

Scikit-learn দিয়ে লজিস্টিক রিগ্রেশন (Logistic Regression)

মেশিন লার্নিং প্র্যাকটিক্যাল নোটস — সহজ বাংলায় ব্যাখ্যা

১. ভূমিকা

লজিস্টিক রিগ্রেশনের গাণিতিক তত্ত্ব (সিগময়েড ফাংশন, কস্ট ফাংশন, গ্র্যাডিয়েন্ট ডিসেন্ট ইত্যাদি) নিজে হাতে কোড করে বাস্তবায়ন করা সম্ভব হলেও, বাস্তব প্রজেক্টে আমরা সাধারণত **Scikit-learn (sklearn)** নামক পাইথন লাইব্রেরি ব্যবহার করি। এটি একটি জনপ্রিয় ও নির্ভরযোগ্য মেশিন লার্নিং লাইব্রেরি, যেখানে জটিল সব গাণিতিক হিসাব-নিকাশ আগে থেকেই অপটিমাইজড কোডে লেখা আছে, ফলে মাত্র কয়েকটি লাইন কোড লিখেই একটি সম্পূর্ণ মডেল তৈরি করা যায়।

এই নোটে আমরা দেখব — **sklearn** দিয়ে কীভাবে ধাপে ধাপে একটি লজিস্টিক রিগ্রেশন মডেল তৈরি, ট্রেনিং, প্রেডিকশন ও মূল্যায়ন (**evaluation**) করতে হয়, সাথে গুরুত্বপূর্ণ প্যারামিটার ও মেথডগুলো নিয়ে আলোচনা করব।

২. প্রয়োজনীয় লাইব্রেরি ইম্পোর্ট করা

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd
```

এখানে **LogisticRegression** হলো মূল মডেল ক্লাস, **train_test_split** ডেটাকে ট্রেনিং ও টেস্ট অংশে ভাগ করার জন্য, **StandardScaler** ফিচার স্কেলিং-এর জন্য, এবং **metrics** মডিউলটি মডেলের পারফরম্যান্স যাচাইয়ের জন্য ব্যবহৃত হয়।

৩. সম্পূর্ণ ওয়ার্কফ্লো (Step-by-Step)

ধাপ ১: ডেটা প্রস্তুত করা

```
# X = ইনপুট ফিচারসমূহ, y = আউটপুট লেবেল (০ অথবা ১)
X = df[['feature1', 'feature2', 'feature3']]
y = df['target']
```

এখানে **X**-তে থাকবে সব ইনপুট ফিচার (independent variables), আর **y**-তে থাকবে যে ক্লাস প্রেডিক্ট করতে চাই (dependent variable), যেমন ০ (না) অথবা ১ (হ্যাঁ)।

ধাপ ২: ট্রেনিং ও টেস্ট ডেটা ভাগ করা

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

ডেটাসেটকে সাধারণত ৮০% ট্রেনিং ও ২০% টেস্টে ভাগ করা হয় (**test_size=0.2**)। **random_state** ব্যবহার করলে প্রতিবার একই রকম ভাগ পাওয়া যায় (reproducibility)।

ধাপ ৩: ফিচার স্কেলিং করা

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

গুরুত্বপূর্ণ: যদিও লজিস্টিক রিগ্রেশন KNN-এর মতো পুরোপুরি দূরত্ব-নির্ভর নয়, তবুও ফিচারগুলোর মানের পরিসর (scale) খুব ভিন্ন হলে গ্র্যাডিয়েন্ট ডিসেন্ট ভিত্তিক অপটিমাইজেশন ধীরগতির হতে পারে বা ঠিকমতো কনভার্জ (converge) নাও করতে পারে। তাই স্কেলিং করে নেওয়া ভালো অভ্যাস। লক্ষণীয়, **fit_transform()** শুধু ট্রেনিং ডেটায় করা হয়, আর টেস্ট ডেটায় শুধু **transform()** — যাতে টেস্ট ডেটার তথ্য মডেল ট্রেনিং-এ "লীক" (data leakage) না হয়।

ধাপ ৪: মডেল তৈরি ও ট্রেনিং করা

```
model = LogisticRegression()
model.fit(X_train_scaled, y_train)
```

LogisticRegression() দিয়ে একটি মডেল অবজেক্ট তৈরি করা হয়, আর **.fit()** মেথড দিয়ে সেই মডেলকে ট্রেনিং ডেটা দিয়ে প্রশিক্ষণ দেওয়া হয় — এই ধাপেই ভেতরে ভেতরে গ্র্যাডিয়েন্ট ডিসেন্ট বা অন্য কোনো অপটিমাইজেশন পদ্ধতি চালিয়ে সবচেয়ে ভালো **w** (weight) ও **b** (bias) খুঁজে বের করা হয়।

ধাপ ৫: প্রেডিকশন করা

```
y_pred = model.predict(X_test_scaled) # সরাসরি ক্লাস (০ বা ১)
y_pred_proba = model.predict_proba(X_test_scaled) # সম্ভাবনা
```

- `.predict()` — সরাসরি চূড়ান্ত ক্লাস (০ অথবা ১) রিটার্ন করে (থ্রেশহোল্ড ০.৫ ব্যবহার করে)।
- `.predict_proba()` — প্রতিটি ক্লাসের সম্ভাবনা (probability) রিটার্ন করে, যেমন `[0.23, 0.77]` — অর্থাৎ ক্লাস ১ হওয়ার সম্ভাবনা ৭৭%।

ধাপ ৬: মডেলের পারফরম্যান্স যাচাই করা

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Report:\n", classification_report(y_test, y_pred))
```

- accuracy_score:** মোট প্রেডিকশনের মধ্যে কতগুলো সঠিক হয়েছে তার শতকরা হার।
- confusion_matrix:** সঠিক ও ভুল প্রেডিকশনের বিস্তারিত হিসাব (True Positive, False Positive, ইত্যাদি) একটি টেবিল আকারে দেখায়।
- classification_report:** Precision, Recall, F1-score-সহ বিস্তারিত রিপোর্ট দেখায়, যা শুধু accuracy-এর চেয়ে অনেক বেশি তথ্যবহুল, বিশেষত ইমব্যালান্সড ডেটাসেটে।

৪. গুরুত্বপূর্ণ প্যারামিটার (Important Parameters)

`LogisticRegression()` তৈরি করার সময় বেশ কিছু গুরুত্বপূর্ণ প্যারামিটার সেট করা যায়:

প্যারামিটার	কাজ
<code>penalty</code>	রেগুলারাইজেশনের ধরন — <code>'l1'</code> , <code>'l2'</code> (ডিফল্ট), <code>'elasticnet'</code> , বা <code>'none'</code> , যা ওভারফিটিং কমাতে সাহায্য করে
<code>C</code>	রেগুলারাইজেশনের শক্তি নিয়ন্ত্রণ করে — মান যত ছোট, রেগুলারাইজেশন তত বেশি (ডিফল্ট = ১.০)
<code>solver</code>	অপটিমাইজেশন অ্যালগরিদম বেছে নেয় — যেমন <code>'lbfgs'</code> (ডিফল্ট), <code>'liblinear'</code> , <code>'saga'</code>
<code>max_iter</code>	মডেল কনভার্জ (converge) না হলে সর্বোচ্চ কতবার iteration চালানো হবে (ডিফল্ট = ১০০)
<code>multi_class</code>	একাধিক ক্লাস থাকলে কোন কৌশল ব্যবহার হবে — <code>'ovr'</code> বা <code>'multinomial'</code>
<code>random_state</code>	ফলাফল পুনরুৎপাদনযোগ্য (reproducible) করার জন্য একটি নির্দিষ্ট মান সেট করা

উদাহরণ:

```
model = LogisticRegression(
    penalty='l2',
    C=0.5,
    solver='lbfgs',
    max_iter=200,
    random_state=42
)
```

এখানে `C=0.5` মানে তুলনামূলক শক্তিশালী রেগুলারাইজেশন প্রয়োগ করা হচ্ছে, যা মডেলকে অতিরিক্ত জটিল হয়ে ওভারফিট করা থেকে বিরত রাখতে সাহায্য করবে।

৫. মডেলের গুরুত্বপূর্ণ অ্যাট্রিবিউট (Attributes)

মডেল ট্রেনিং হয়ে গেলে (`.fit()` করার পর) নিচের অ্যাট্রিবিউটগুলো থেকে গুরুত্বপূর্ণ তথ্য পাওয়া যায়:

```
print(model.coef_) # প্রতিটি ফিচারের ওজন (weight)
print(model.intercept_) # বায়াস (intercept/bias) মান
```

`model.coef_` থেকে বোঝা যায় প্রতিটি ফিচার প্রেডিকশনের উপর কতটা এবং কোন দিকে (ধনাত্মক/ঋণাত্মক) প্রভাব ফেলছে — এটিই লজিস্টিক রিগ্রেশনের অন্যতম বড় সুবিধা, কারণ এই সংখ্যাগুলো দেখেই মডেলের সিদ্ধান্ত ব্যাখ্যা করা যায়।

৬. থ্রেশহোল্ড পরিবর্তন করা (Custom Threshold)

ডিফল্টভাবে `.predict()` ০.৫ থ্রেশহোল্ড ব্যবহার করে, কিন্তু প্রয়োজনে `predict_proba()` থেকে পাওয়া সম্ভাবনা ব্যবহার করে নিজের মতো থ্রেশহোল্ড বসানো যায়:

```
probs = model.predict_proba(X_test_scaled)[: , 1]
custom_pred = (probs >= 0.3).astype(int) # থ্রেশহোল্ড ০.৩
```

এটি বিশেষভাবে দরকারি হয় যেমন মেডিকেল ডায়াগনসিসে, যেখানে রোগ মিস করার (False Negative) ঝুঁকি কমাতে থ্রেশহোল্ড কম রাখা যেতে পারে।

৭. Scikit-learn ব্যবহার করার সুবিধা

- ✓ খুব কম কোডে সম্পূর্ণ মডেল:
ম্যানুয়ালি গ্র্যাডিয়েন্ট ডিসেন্ট কোড না করেই মাত্র কয়েক লাইনে ট্রেনিং সম্পন্ন করা যায়।
- ✓ অপটিমাইজড ও দ্রুত:
ভেতরে ভেতরে দক্ষ (efficient) অ্যালগরিদম ব্যবহার করে, যা নিজে কোড করার চেয়ে অনেক দ্রুত কাজ করে।
- ✓ রেগুলারাইজেশন বিস্ট-ইন:
L1/L2 রেগুলারাইজেশন আগে থেকেই যুক্ত থাকে, যা ওভারফিটিং কমাতে সাহায্য করে।
- ✓ সহজে মূল্যায়ন করা যায়:
accuracy, confusion matrix, precision-recall-এর মতো মেট্রিক্স আলাদাভাবে হাতে কোড করতে হয় না।
- ✓ অন্যান্য টুলের সাথে সহজে যুক্ত হয়:
pandas, numpy, matplotlib-এর সাথে খুব সহজেই কাজ করা যায়।

৮. Scikit-learn ব্যবহার করার সীমাবদ্ধতা

- ✗ ভেতরের হিসাব-নিকাশ "ব্ল্যাক বক্স":
ভেতরে ঠিক কীভাবে গ্র্যাডিয়েন্ট ডিসেন্ট বা অন্য অপটিমাইজেশন কাজ করছে তা সরাসরি দেখা যায় না, তাই মৌলিক ধারণা আলাদাভাবে শেখা জরুরি।
- ✗ ভুল প্যারামিটার বাছাই করলে ফলাফল খারাপ হতে পারে:
যেমন `solver` বা `max_iter` সঠিকভাবে সেট না করলে মডেল ঠিকমতো কনভারজ নাও করতে পারে।
- ✗ বড় ডেটাসেটে কাস্টমাইজেশন সীমিত:
খুব বিশেষায়িত বা কাস্টম লস ফাংশন দরকার হলে TensorFlow/PyTorch-এর মতো লাইব্রেরি বেশি উপযোগী।

৯. সংক্ষিপ্ত সারাংশ (Summary)

- Scikit-learn-এর `LogisticRegression` ক্লাস ব্যবহার করে মাত্র কয়েক লাইনে একটি সম্পূর্ণ লজিস্টিক রিগ্রেশন মডেল তৈরি, ট্রেনিং ও মূল্যায়ন করা যায়।
- মূল ওয়ার্কফ্লো: ডেটা ভাগ করা (`train_test_split`) → স্কেলিং (`StandardScaler`) → ট্রেনিং (`.fit()`) → প্রেডিকশন (`.predict()`) → মূল্যায়ন (`accuracy_score`, `classification_report`)।
- `C` ও `penalty` -এর মতো প্যারামিটার দিয়ে রেগুলারাইজেশন নিয়ন্ত্রণ করা যায়, যা ওভারফিটিং প্রতিরোধ করে।
- `model.coef` ও `model.intercept` থেকে মডেলের সিদ্ধান্ত ব্যাখ্যা করা যায়, যা লজিস্টিক রিগ্রেশনের একটি বড় শক্তি।
- Scikit-learn ব্যবহার করা দ্রুত ও সহজ হলেও, ভেতরের গাণিতিক তত্ত্ব (সিগময়েড, কস্ট ফাংশন, গ্র্যাডিয়েন্ট ডিসেন্ট) বোঝা মডেল সঠিকভাবে ব্যবহার ও ডিবাগ করার জন্য অত্যন্ত গুরুত্বপূর্ণ।